

Machine Learning II - Structured Models

Exercise sheet **Tree-decomposition based algorithms: Subgradient vs. TRW-S.**

Bogdan Savchynskyy - Bogdan.Savchynskyy@tu-dresden.de

Exercise 1 (Synthetic problem). Download the synthetic dataset from the supplementary material. Run

```
./opengm_min_sum --modelinfo -m synthetic200x100x5_amp1.0_s1001.0.h5
```

What is your opinion, which what kind of structure has the graph of this model? Chain? Grid? Is it acyclic? How many labels it has?

The potentials of this model were generated randomly from the uniform distribution on the interval $[0, 1]$. The corresponding energy minimization problem is not LP tight. Moreover, even attaining an optimum of the dual LP problem is not easy, at least for coordinate descent algorithms like **diffusion** or **TRW-S** - they converge to arc-consistency, which is only necessary, but not sufficient for optimality.

In contrast, the **subgradient** method converges to the dual LP optimum, at least in theory. The task of this exercise – empirically find the best step-size schedule for the **subgradient** algorithm.

Before playing with **subgradient**, first find an approximate value of the optimal dual LP. To this end run the smoothing-based **TRW-S** modification (is not covered by the lectures), **ADSa1** (Adaptive Diminishing Smoothing algorithm), which provably converges to the optimum and, moreover, provides an approximate feasible relaxed primal solution as an optimality certificate.

```
./opengm_min_sum -a ADSa1 --maxIt 200 -m synthetic200x100x5_amp1.0_s1001.0.h5 --precision 50 -p 1 -v
```

The energy of the estimated primal relaxed solution is shown as **primalLPbound**.

Run **TRW-S** to get an idea of its convergence.

```
./opengm_min_sum -a TRWS --maxIt 200 -m synthetic200x100x5_amp1.0_s1001.0.h5 -p 1 -v
```

Run the **subgradient** algorithm. If you see that it has numerically got stuck, stop it by pressing **Ctrl-C**:

```
./opengm_min_sum -a DDSubgradient --maxIt 200 -m synthetic200x100x5_amp1.0_s1001.0.h5 -p 1 -v
```

Check parameters of the **subgradient** algorithm, which define its used step-size:

```
./opengm_min_sum -a DDSubgradient -h
```

There are 4 different step-size rules: **ProjectedAdaptive**, **Adaptive**, **StepLength**, **StepSize**. In general they depend on:

1. **primalDualGap** = **value-bound**;

2. **a** - scale of the step-size rule, is regulated by the parameter `--ssScale`;
3. **i** -iteration counter;
4. **e** -exponent of the step-size (e.g. the default square root value is obtained by setting **e**= 0.5), is regulated by the parameter `--ssExponent`;
5. you might also consider to play with the minimal and maximal step-sizes (parameters `--ssMin` and `--ssMax`) and decomposition type (parameter `--decomp`).

Play with parameters with the goal to attain better dual LP bound than those of TRW-S.

Exercise 2 (Color segmentation). Run the `subgradient` method on `crops` and `clownfish`. First do it with the best parameters from the previous exercise. Afterwards try to tune parameters to the given datasets. Compare the obtained iteration-wise convergence speed to those of the TRW-S algorithm.

Exercise 3 (Stereo). Run the `subgradient` method on `tsu-gm` and `ven-gm`. First do it with the best parameters from the previous exercises. Afterwards try to tune parameters to the given datasets. Compare the obtained iteration-wise convergence speed to those of the TRW-S algorithm.

Exercise 4 (Protein folding). Download the datasets 1CKK and 2BE6, run `opengm_min_sum` with the parameter `--modelinfo`. These datasets have a small number of constraints, but their graph is dense and the number of labels is big. The LP relaxation is not tight and coordinate ascent algorithms like TRW-S get stuck in suboptimal points (satisfying arc-consistency). Play with parameters of the subgradient to get better results than those of TRW-S. You can use output of the ADSal algorithm as a reference value.